

Contents

SQL Result Validation Prompt Chain	1
Use Case	1
Required Local Preparation	1
Prompt Sequence	1
Related Reference Drawers	2
Relationship To Existing Flows	2
SQL Result Validation Chain - 01 Query And Intent Intake	2
SQL Result Validation Chain - 02 Logic And Grain Review	3
SQL Result Validation Chain - 03 Row Count And Duplicate Checks	4
SQL Result Validation Chain - 04 CTE And Join Checks	5
SQL Result Validation Chain - 05 Final Validation Report	6

SQL Result Validation Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to validate that Oracle SQL results are correct, not merely that the SQL parses or runs.

The workflow is for result correctness review after a query exists or while a query is being finalized. It forces Copilot to propose checks the analyst can run locally for output grain, row counts, duplicates, required-field nulls, aggregate reconciliation, CTE-level counts, join row multiplication, and date-window boundaries.

Copilot does not have database access. It must propose validation SQL or manual checks for the analyst to run in the local database environment, then interpret the analyst-provided evidence.

Required Local Preparation

- Have the current Oracle SQL ready, including all CTEs, joins, filters, and final output columns.
- Prepare the business intent, expected output grain, required fields, key dimensions, metric definitions, date-window rules, and acceptance criteria.
- Gather compact schema context for source tables, keys, join columns, date fields, required non-null fields, and known one-to-one or one-to-many relationships.
- Keep local run results available as the chain progresses, such as row counts, duplicate checks, null counts, reconciliation totals, CTE-level counts, join-step counts, boundary-date samples, or old-vs-new comparison evidence.

Prompt Sequence

1. `01-query-and-intent-intake.md`: provide the SQL, business intent, expected grain, key dimensions, required fields, metrics, date window, and available evidence.

2. `02-logic-and-grain-review.md`: require Copilot to map the result grain, logic, required columns, metric definitions, filters, and unresolved assumptions before generating checks.
3. `03-row-count-and-duplicate-checks.md`: generate local checks for final output grain, row counts by key dimensions, duplicate keys, required-field nulls, and date-window boundaries.
4. `04-cte-and-join-checks.md`: generate local checks for CTE-level row counts, CTE grain, join row multiplication, unmatched joins, and aggregate reconciliation.
5. `05-final-validation-report.md`: convert the analyst-run evidence into a final validation report table and decision.

Related Reference Drawers

- `references/oracle-sql/sections/cte_patterns.md`
- `references/oracle-sql/sections/join_patterns.md`
- `references/oracle-sql/sections/aggregation_patterns.md`

Use these drawers when detailed CTE, join, or aggregation guidance is needed. Do not duplicate long SQL tutorial content from the drawers in Copilot prompt responses; use them as local references for designing focused validation checks.

Relationship To Existing Flows

- Use `prompt-chains/sql-general/` first when the request is broad or when the analyst still needs help classifying, drafting, or explaining the SQL task.
- Use this chain after `sql-general` when the SQL exists and the main question is whether the returned results are correct.
- Use `prompt-chains/sql-cte-validation/` when the primary need is detailed CTE mapping or CTE refactor review. Use this result-validation chain when the CTE checks must roll into final result correctness evidence.
- Use `prompt-chains/sql-joins/` when the primary need is choosing or explaining join types. Use this chain when join behavior must be tested for dropped rows, duplicated rows, or row multiplication in the final output.
- Use `prompt-chains/sql-aggregation/` when the primary need is metric and grouping design. Use this chain when aggregate totals must be reconciled against source data or acceptance criteria.
- Use `prompt-chains/query-tuning/` after result correctness is established and the main problem is runtime, plan evidence, indexing, or optimizer behavior.

SQL Result Validation Chain - 01 Query And Intent Intake

Act as an Oracle SQL result validation reviewer.

This workflow validates result correctness, not just SQL syntax. Do not rewrite or optimize the query yet. Copilot does not have database access, so propose checks that I can run locally and ask for evidence I need to collect.

Current Oracle SQL:

<paste **the full** Oracle SQL **query** here, **including** CTEs, joins, filters, **and** final **SELECT**>

Business intent:

<paste the reporting goal, business rule, expected result, or acceptance criteria here>

Expected output grain:

<example: one row per customer per month; one row per policy; one row per account/status/date>

Key dimensions and expected row-count behavior:

<paste output keys, grouping dimensions, row-count expectations, and any known totals by segment>

Required output fields and null rules:

<paste fields that must be populated, fields allowed to be null, and why>

Metric definitions and reconciliation targets:

<paste definitions for counts, sums, distinct counts, rates, source totals, or old-vs-new totals>

Date-window rules:

<paste date fields, start/end dates, inclusive/exclusive boundaries, reporting periods, and time windows>

Known schema context:

<paste source tables, primary keys, foreign keys, natural keys, join notes, date fields, and other schema details>

Available validation evidence:

<paste any row counts, duplicate counts, null counts, reconciliation totals, samples, or other evidence>

Return:

1. Inferred business objective.
2. Expected final output grain and whether it is explicit or inferred.
3. Candidate output key columns for duplicate checks.
4. Required output fields that need null checks.
5. Key dimensions for row-count checks.
6. Metrics that need aggregate reconciliation.
7. CTEs, joins, and date filters that appear validation-sensitive.
8. Missing context or evidence required before result correctness can be validated.
9. A short local-evidence request list for the next prompt.

Point me to `references/oracle-sql/sections/cte_patterns.md`, `references/oracle-sql/sections/join_patterns.md`, or `references/oracle-sql/sections/aggregation_patterns.md` where relevant, but do not repeat long tutorial content from those drawers.

SQL Result Validation Chain - 02 Logic And Grain Review

Continue the Oracle SQL result validation workflow.

Use the intake summary to review result logic and output grain before producing validation SQL. This is not a syntax-only review and not a rewrite request. Copilot does not have database access; identify checks I should run locally.

Intake summary:

<paste Copilot's intake summary from prompt 01>

Current Oracle SQL:

<paste **the current** Oracle SQL again **if needed**>

Additional local context or evidence:

<paste answers to missing-context questions, sample rows, row counts, schema notes, or acceptance criteria>

Return:

1. Final output grain statement in one sentence.
2. Grain risk review:
 - expected key columns;
 - where the SQL could create more than one row per expected key;
 - where the SQL could collapse rows too early.
3. Logic map from source data to final output:
 - source tables or CTEs;
 - filters;
 - joins;
 - aggregations;
 - derived fields;
 - final output fields.
4. Required-field null risk list.
5. Aggregate reconciliation risk list.
6. Date-window boundary risk list, including inclusive and exclusive boundary assumptions.
7. CTE and join stages that need row-count or row-multiplication checks.
8. A validation plan outline, without full SQL yet, grouped into:
 - output grain and duplicate checks;
 - row counts by key dimensions;
 - required-field null checks;
 - aggregate reconciliation checks;
 - CTE-level checks;
 - join row multiplication checks;
 - date-window boundary checks.

If the query uses CTE-heavy logic, point me to [references/oracle-sql/sections/cte_patterns.md](#).

If join cardinality is a major risk, point me to [references/oracle-sql/sections/join_patterns.md](#).

If grouped metrics or totals are involved, point me to [references/oracle-sql/sections/aggregation_patterns.md](#).

Do not duplicate long tutorial content from those drawers.

SQL Result Validation Chain - 03 Row Count And Duplicate Checks

Continue the Oracle SQL result validation workflow.

Generate focused Oracle SQL or manual checks I can run locally for final-output correctness. Copilot does not have database access, so do not claim any check passes until I provide results.

Current query or final-output CTE:

<paste **the current** Oracle SQL, **or** wrap **the final query as** a CTE **named** final_result>

Accepted output grain and key columns:

<paste expected grain and key columns from prompt 02>

Key dimensions for row-count checks:

<paste dimensions such as period, product, status, region, source system, customer type>

Required output fields:

<paste fields that should not be null, plus fields where null is acceptable>

Date-window rules:

<paste date field, start date, end date, inclusive/exclusive boundary rules, and reporting period>

Return validation checks for me to run locally:

1. Final output total row count.
2. Row counts by each key dimension and important dimension combinations.
3. Duplicate-key check for the expected output grain.
4. Null checks for all required output fields.
5. Date-window boundary checks:
 - rows before the expected start boundary;
 - rows on the start boundary;
 - rows on the end boundary;
 - rows after the expected end boundary;
 - suspicious time components if date truncation matters.
6. Small sample-row checks for duplicate keys, null required fields, and boundary dates.
7. Expected pass/fail signal for each check.
8. How I should paste the local results back for interpretation.

Use small Oracle SQL snippets that inspect the final result. If wrapping the query in a CTE is useful, use a placeholder like:

```
WITH final_result AS (  
  <paste current query here>  
)  
SELECT ...  
FROM final_result;
```

Keep the snippets focused on validation, not final rewrites.

SQL Result Validation Chain - 04 CTE And Join Checks

Continue the Oracle SQL result validation workflow.

Generate focused Oracle SQL or manual checks I can run locally for intermediate CTEs, joins, and aggregates. Copilot does not have database access, so do not claim any check passes until I provide results.

Current Oracle SQL:

<paste the full query with CTEs, joins, filters, aggregations, and final SELECT>

Output grain and validation plan:

<paste the accepted output grain and validation plan from prompt 02>

Results from prompt 03, if available:

<paste final-output row counts, duplicate counts, null checks, and boundary checks>

Known relationship and metric context:

<paste known primary keys, foreign keys, one-to-many relationships, bridge tables, metric definitions>

Return validation checks for me to run locally:

1. CTE-level row counts in dependency order.
2. CTE-level grain or duplicate checks for each stage where a key is expected.
3. CTE-level null checks where required fields are introduced or transformed.
4. Filter-impact checks where a CTE applies business filters.
5. Join row multiplication checks:
 - pre-join counts by join key;
 - duplicate join-key counts on each side;
 - post-join counts by join key;
 - unmatched rows for outer joins;
 - many-to-many risk checks.
6. Aggregate reconciliation checks:
 - source total versus final total;
 - source count versus final count;
 - grouped totals by key dimensions;
 - distinct-count checks where duplicate amplification is possible.
7. Expected pass/fail signal for each check.
8. How I should paste local results back for final report assembly.

Point me to [references/oracle-sql/sections/cte_patterns.md](#) for CTE-stage validation patterns, [references/oracle-sql/sections/join_patterns.md](#) for join-cardinality checks, and [references/oracle-sql/sections/aggregation_patterns.md](#) for reconciliation patterns when relevant. Do not repeat long SQL tutorial content from those drawers.

Keep all SQL snippets focused on validation evidence, not final rewrites or performance tuning.

SQL Result Validation Chain - 05 Final Validation Report

Finish the Oracle SQL result validation workflow.

Use the analyst-run evidence to decide whether the result set is validated, blocked by missing evidence, or likely incorrect. This is result correctness review, not a syntax-only review. Copilot does not have database access; base the conclusion only on the evidence I paste below.

Current Oracle SQL or query version reviewed:

<paste the final SQL version or query identifier here>

Business intent and expected output grain:

<paste the accepted business objective, expected grain, key columns, required fields, metrics, and other context>

Validation checks Copilot proposed:

<paste checks from prompts 03 and 04, or a summarized check list>

Local results from the analyst:

<paste row counts, duplicate checks, null checks, CTE counts, join checks, aggregate reconciliation>

Return:

1. Final decision:

- validated;
- validated with caveats;
- blocked by missing evidence;
- likely incorrect.

2. A validation report table with exactly this shape:

check	purpose	SQL/manual step	expected signal	issue if failed
-------	---------	-----------------	-----------------	-----------------

3. Evidence interpretation for:

- output grain;
- row counts by key dimensions;
- duplicate keys;
- nulls in required output fields;
- aggregate reconciliation;
- CTE-level row counts;
- join row multiplication;
- date-window boundaries.

4. Any failed checks, unclear signals, or missing local evidence.

5. Minimal recommended next action for each failure or evidence gap.

6. Whether the query should move to **prompt-chains/query-tuning/** only after result correctness is established.

If CTE, join, or aggregation details need deeper review, point me to the relevant drawer:

- [references/oracle-sql/sections/cte_patterns.md](#)
- [references/oracle-sql/sections/join_patterns.md](#)
- [references/oracle-sql/sections/aggregation_patterns.md](#)

Do not duplicate long SQL tutorial content from those drawers.